

Titan Engine APIs & Contracts (Implementation Guide)

Purpose

This document defines the public contracts of each Titan engine.

It specifies:

what each engine owns and is allowed to write

the service/API methods each engine must expose

the signals each engine must emit (minimum set)

what other engines are allowed to call

This is the implementation guide that prevents engines from bleeding into each other.

Global Contract Rules

1) Tenant Boundary

All `tz_*` and `ext_*` writes must include `team_id`. All engine APIs must enforce tenant scoping automatically.

2) Write Boundaries

Each engine writes only its domain tables. No engine is allowed to directly mutate another engine's domain tables.

3) Signals Are Mandatory

Lifecycle transitions must emit signals via Titan Signal Engine. Signals are append-only.

4) Pulse Executes Through APIs

Titan Pulse Engine must execute actions by calling Engine APIs, not by writing arbitrary tables.

5) Zero Proposes, Never Writes State

Titan Zero Engine can:

propose actions

request approvals

generate artifacts But it never writes operational state directly.

Engine Contracts

Titan Identity Engine

Owns

Platform tables (no prefix):

users, teams, team_users, roles, permissions, user_devices

Exposes (minimum)

resolveTenant(user_id) → team_id

assertPermission(user_id, ability, team_id)

currentActorContext() → {user_id, team_id, role_ids, permissions}

deviceRegister(user_id, device_fingerprint)

impersonateStart(admin_id, target_user_id) (optional)

Emits Signals (minimum)

identity.user.created

identity.user.updated

identity.team.created

identity.role.changed

Titan Customer Engine

Owns (tz_)

tz_customers*, tz_sites*

Exposes (minimum)

customer.create(data)

customer.update(customer_id, data)

customer.get(customer_id)

customer.search(filters)

site.create(customer_id, data)

site.update(site_id, data)

site.get(site_id)

site.list(customer_id)

Emits Signals (minimum)

customer.created

customer.updated

customer.archived

site.created

site.updated

Titan Work Engine

Owns (tz_)

tz_jobs*, tz_job_visits*, tz_tasks*, tz_checklists*, assignments/events satellites

Exposes (minimum)

job.create(data)

job.update(job_id, data)

job.assign(job_id, user_id)

job.schedule(job_id, scheduled_at)

job.start(job_id)

job.complete(job_id, completion_payload)

job.cancel(job_id, reason)

job.get(job_id)

job.list(filters)

visit.create(job_id, data)

visit.start(visit_id)

visit.complete(visit_id, payload)

task.create(job_id|visit_id, data)

task.complete(task_id, payload)

checklist.apply(job_id|visit_id, checklist_template_id)

checklist.respond(checklist_id, responses)

Emits Signals (minimum)

work.job.created

work.job.assigned

work.job.scheduled

work.job.started

work.job.completed

work.job.cancelled

work.visit.created

work.visit.started

work.visit.completed

work.task.created

work.task.completed

work.checklist.applied

work.checklist.completed

Titan Evidence Engine

Owns (tz_)

tz_evidence_chain, tz_evidence_items, tz_evidence_links

Exposes (minimum)

evidence.upload(entity_type, entity_id, file, meta)

evidence.link(evidence_id, entity_type, entity_id)

evidence.list(entity_type, entity_id)

evidence.verifyChain(evidence_id) (optional)

evidence.bundle(entity_refs[]) → bundle_id (optional; can be Docs Engine)

Emits Signals (minimum)

evidence.uploaded

evidence.linked

evidence.chain.verified (optional)

Titan Docs Engine

Owns (tz_)

tz_documents, tz_document_versions, tz_document_links

Exposes (minimum)

doc.create(type, title, body|template_ref, meta)

doc.version(document_id, body, meta)

doc.link(document_id, entity_type, entity_id)

doc.get(document_id)

doc.render(document_id, format) (pdf/html)

doc.bundle(entity_refs[]) → document_id (evidence pack / job pack)

Emits Signals (minimum)

docs.created

docs.versioned

docs.linked

docs.bundled

Titan Money Engine

Owns (tz_)

tz_quotes*, tz_invoices*, tz_payments*, tz_financial_events*

Exposes (minimum)

quote.create(customer_id, items, meta)

quote.send(quote_id, channel)

quote.accept(quote_id)

quote.reject(quote_id, reason)

quote.convertToJob(quote_id) → job_id (calls Work Engine)

invoice.createFromJob(job_id) → invoice_id

invoice.create(customer_id, items, meta)

invoice.send(invoice_id, channel)

invoice.markPaid(invoice_id, payment_payload)

invoice.markOverdue(invoice_id) (usually automation)

refund.request(invoice_id, amount, reason)

refund.approve(refund_id) (governance gated)

refund.execute(refund_id)

payment.record(invoice_id, amount, method, meta)

Emits Signals (minimum)

finance.quote.created

finance.quote.sent

finance.quote.accepted

finance.quote.rejected

finance.invoice.created

finance.invoice.sent

finance.invoice.overdue

finance.invoice.paid

finance.refund.requested

finance.refund.approved

finance.refund.executed

finance.payment.recorded

Titan Talk Engine

Owns (tz_)

tz_threads, tz_messages, tz_message_attachments, tz_thread_links, tz_contacts

Exposes (minimum)

thread.create(participants, meta)

thread.link(thread_id, entity_type, entity_id)

thread.get(thread_id)

thread.list(filters)

message.send(thread_id, body, channel, attachments?)

message.receive(external_payload) (ingestion endpoint)

message.list(thread_id)

message.attach(message_id, file)

Emits Signals (minimum)

talk.thread.created

talk.thread.linked

talk.message.sent

talk.message.received

talk.message.failed (optional)

Titan Signal Engine

Owns (tz_)

tz_signals, tz_signal_events, tz_signal_consumers

Exposes (minimum)

signal.emit(event_type, entity_type, entity_id, payload, meta)

signal.subscribe(consumer_name, event_patterns, handler_ref)

signal.replay(filters, consumer_name?)

signal.ack(signal_id, consumer_name) (if using consumer offsets)

Emits Signals

Signal Engine is the emitter; it does not emit about itself except optional:

signal.consumer.failed

signal.consumer.lagging

Titan Pulse Engine

Owns (tz_)

tz_automation_rules, tz_automation_triggers, tz_automation_runs, tz_automation_logs

Exposes (minimum)

rule.create(trigger_event, conditions, actions, priority)

rule.enable(rule_id)

rule.disable(rule_id)

rule.test(rule_id, sample_signal_id)

run.execute(signal_id) (internal)

run.retry(run_id)

run.cancel(run_id)

monitor.health()

Action Types (minimum)

Actions must resolve to Engine API calls, e.g.:

work.job.assign

money.invoice.createFromJob

talk.message.send

docs.bundle

evidence.link

Emits Signals (minimum)

automation.triggered

automation.executed

automation.failed

automation.escalated (to Titan Zero)

Titan Zero Engine

Owns

Governance + AI metadata tables (tz_ preferred) such as:

tz_governance_decisions

tz_ai_requests

tz_ai_artifacts

tz_ai_routing_logs

tz_ai_assistant_profiles

Exposes (minimum)

assist.invoke(assistant_id, context_envelope)

assist.proposeActions(context) → proposals[]

govern.evaluate(proposal, risk_level) → decision

router.selectModel(task_type, risk_level, modalities) → model_ref

artifact.generate(type, inputs) → artifact_ref

approval.queue(list)

approval.resolve(approval_id, decision)

Governance Modes

Default: Titan Zero Mode (monitoring + guidance)

Risk-triggered: Trinary Bayesian Mode (Omega + Omicron + Core)

Emits Signals (minimum)

zero.proposal.created

zero.proposal.approved

zero.proposal.rejected

zero.risk.escalated

zero.artifact.generated

Titan Analytics Engine

Owns (tz_)

tz_metrics, tz_metric_snapshots, tz_analytics_queries

Exposes (minimum)

metric.define(name, definition)

metric.ingest(signal_id)

snapshot.generate(period)

dashboard.query(query_def)

forecast.run(metric, horizon) (optional)

Emits Signals (optional)

analytics.snapshot.created

analytics.alert.triggered

Standard Signal Namespaces (Canonical)

identity.*

customer.*

work.*

evidence.*

docs.*

finance.*

talk.*

automation.*

zero.*

analytics.*

Integration Rules for Extensions

Extensions must:

write only ext_* tables

emit signals through Titan Signal Engine

trigger system behaviour via Pulse rules or Engine API calls

avoid direct writes into tz_* except through Engine APIs